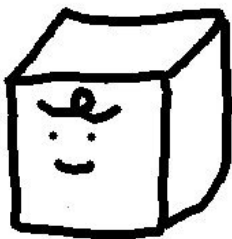


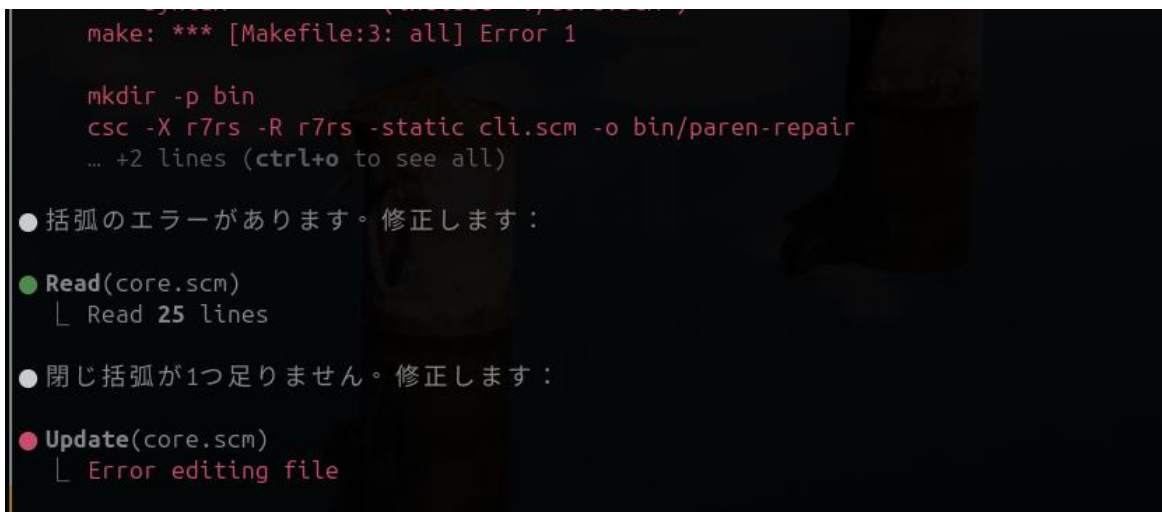
壊れたカッコを直すプログラム



Niyarin

LLMが壊したカッコを直したい

- LLMはカッコの対応を誤って出すことがある
 - 最近改善したように感じるが、また改悪するかもしれない
 - LLMはそもそもカッコのカウントが苦手



```
make: *** [Makefile:3: all] Error 1

mkdir -p bin
csc -X r7rs -R r7rs -static cli.scm -o bin/paren-repair
... +2 lines (ctrl+o to see all)
```

- 括弧のエラーがあります。修正します：
- Read(core.scm)
 - └ Read 25 lines
- 閉じ括弧が1つ足りません。修正します：
- Update(core.scm)
 - └ Error editing file

- カッコの修正ループも厄介

どのように直すのか

- インデント情報ベース

```
(cons (cons a b)  
      c)
```

改行が入ったから閉カッコが入るはず

- 構文情報ベース

```
(cons (cons a b)  
      c)
```

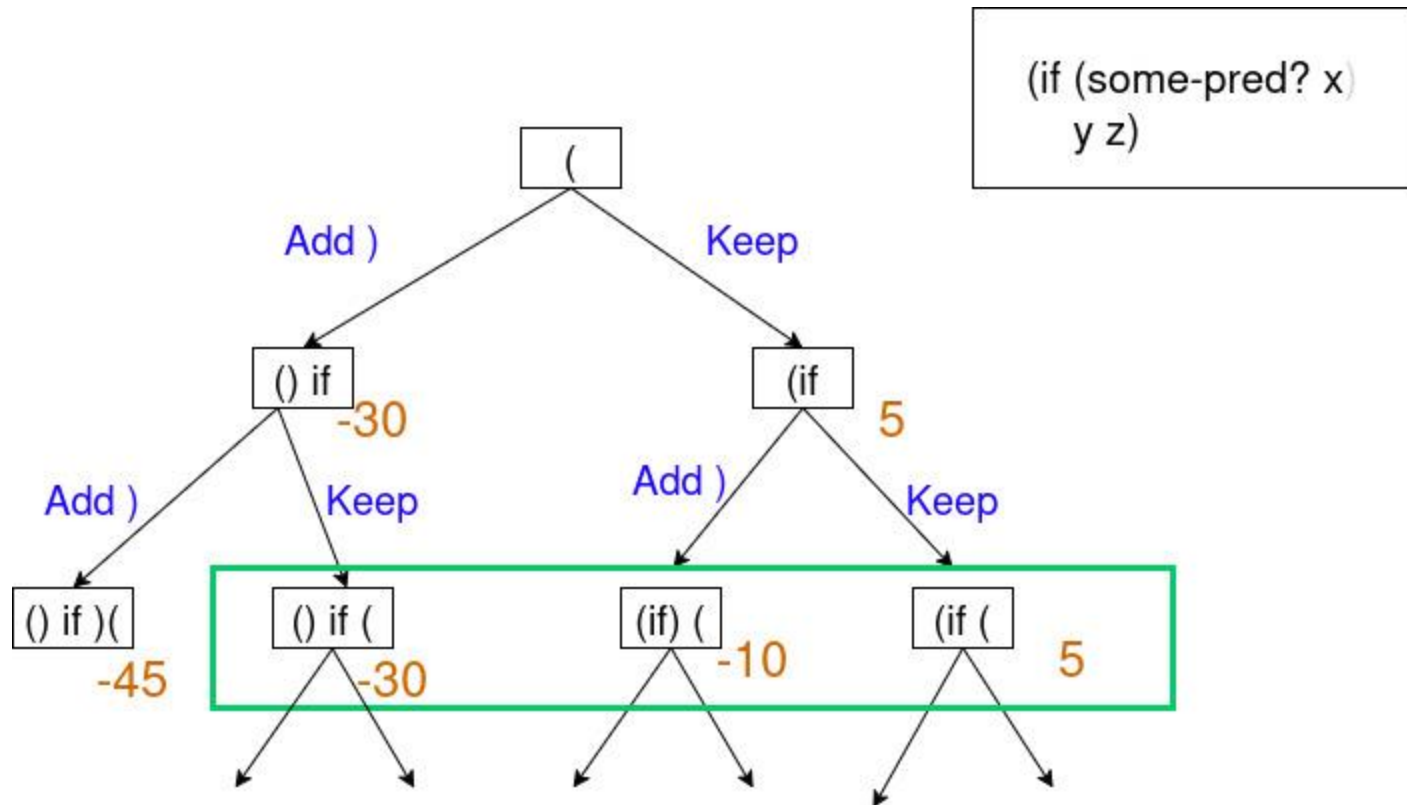
consの引数は2つなので閉じカッコが入るはず

作ったもの

-  niyarin/paren-repair
 - R{5,6,7}RS Scheme用
 - 開カッコの壊れは現状かんがえられていない
- カッコの挿入/削除の各パターンに対してヒューリスティック探索
 - ビームサーチでいくつかの指標から評価
 - インデントによるスコア
 - カッコの対応によるスコア
 - 構文チェックによるスコア
 - PDA (pushdown automaton)をつかう

ビームサーチの例

- 探索木の各深さでスコアが上位N個のものだけをとる



スコアリング方法

- 操作スコア

- KEEP : そのままなにもしない + 0
- ADD) : 閉じカッコを足す -3
- DELTETE) : 閉じカッコを消す -1

- バランス スコア

- 閉じカッコ時にカッコがバランスしていれば+10
- 過剰なカッコのときは-10

- インデントスコア

- 次のページで説明する

- 構文スコア

- 不正な構文だと -30

スコアリング方法 (インデント)

- 最寄りの開きカッコの位置と前要素の位置を使う
 - 開きカッコより左側にくるとペナルティ -90

```
(foo  
bar
```

- 前要素と同じ行 or 列だと加点+6

```
(foo bar  
  baz
```

```
(foo bar baz
```

- 2,3,4番めの要素以外でのズレ-50.0

```
(aaa bbb ccc ddd  
  eee
```

スコアリング方法 (インデント) 続き

- 2,3,4番めの要素のずれは 低減点 -1
 - こういうのを許容するため

```
(foo  
  bar  
  baz
```

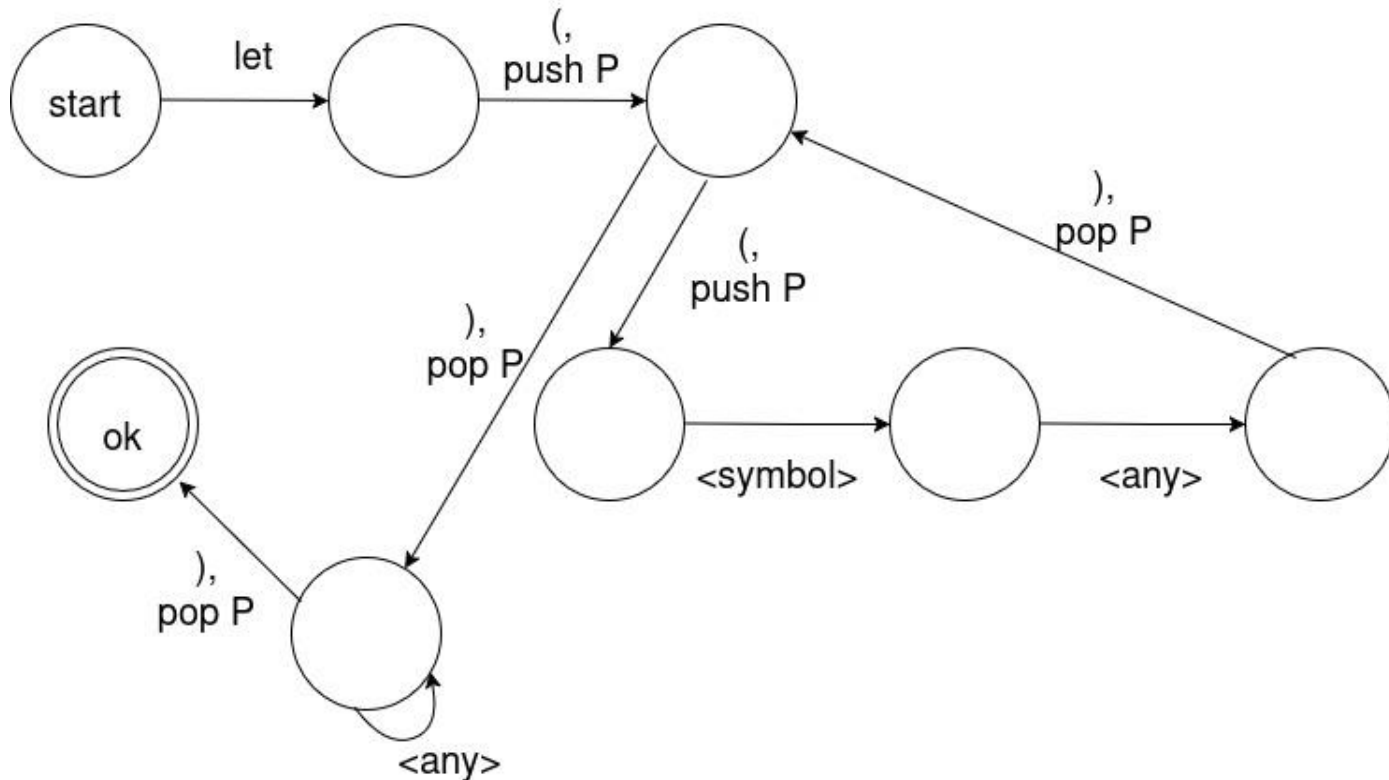
```
(let ((a b))  
  CCC
```

- ただし 3,4 番めの右ずれは中減点 (-20)

```
(let ((a b))  
  CCC
```


PDAベースの構文チェッカ

- 遷移できない状態になったら構文エラー
- スタック付きの有限オートマトン
 - Stack topの内容と入力で状態遷移する

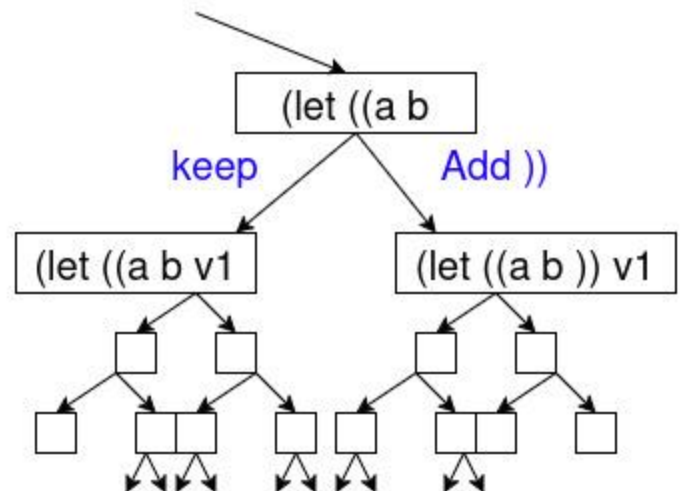


PDAベースの構文チェッカの利点

- 利点:
 - 構文の途中までで構文が壊れているか確認できること
- ナイーブな実装の場合
 - 構文開始に対応する閉じカッコが来たらリストにして構文チェックする
 - 長い不正な式が高スコアのまま生き残り続けることがある

(let ((a b **v1**)) v2 v3 v9999)

PDAだとここで減点できる



ナイーブだとここで減点



まとめ

- 壊れたカッコを修正するプログラムを解説した
 - コードをスコアリングし、ビームサーチで枝刈り
 - スコアリング手法
 - インデント
 - カッコの対応
 - 構文の正しさ
- PDAを使った構文チェッカ
- 余力があればやりたいこと
 - LLMとの連携